

Kargamine — Modules, Flow & Data Map

A full map of this Laravel application: every module and the controller functions behind it, the end-to-end business process from first contact to delivered cargo, and how data moves between them. Built directly from the live routes, controllers, and models — not aspirational.

15

MODULES

62

CONTROLLERS

~100

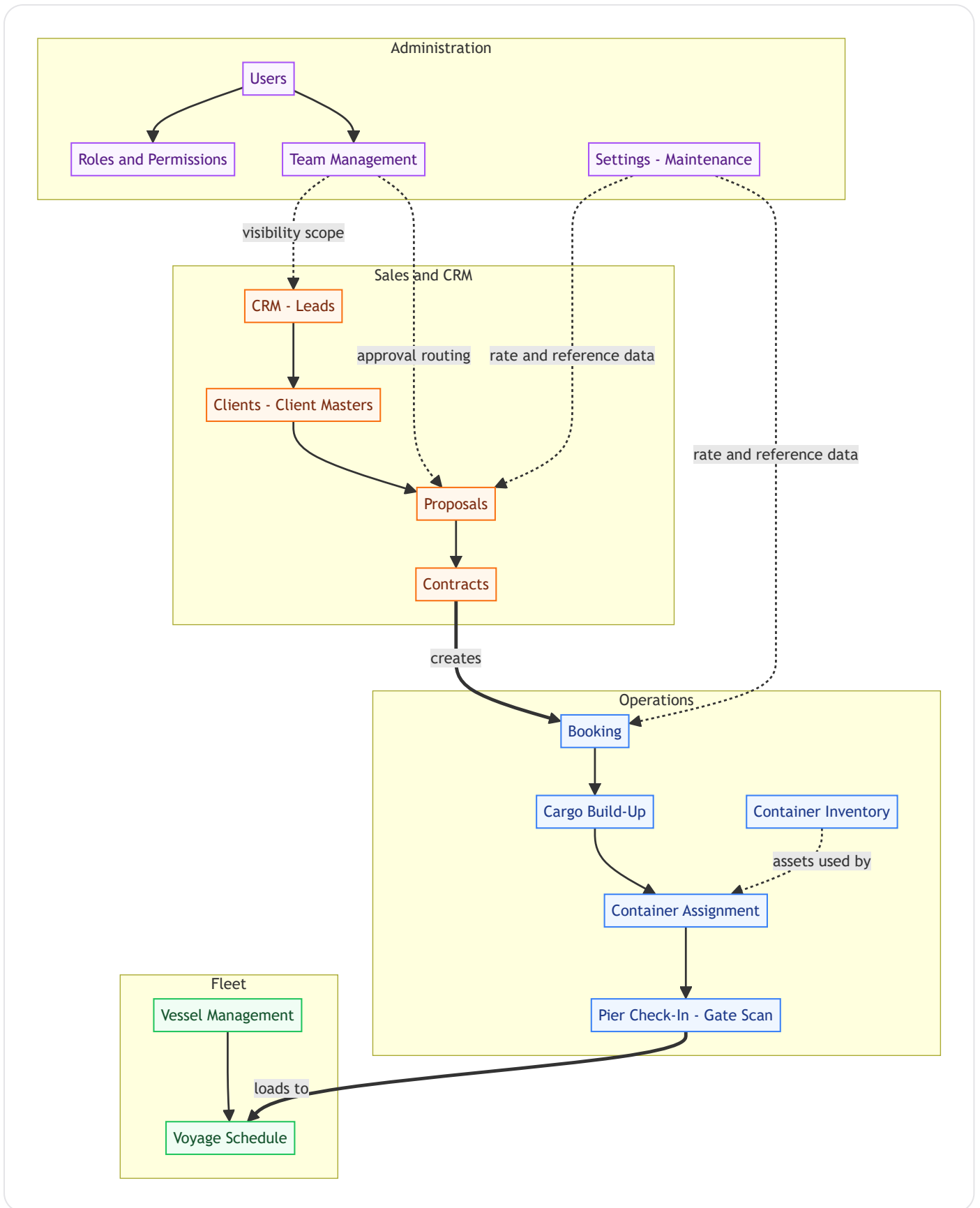
MODELS

4

ORPHANED ROUTE FILES

1 • Module Map

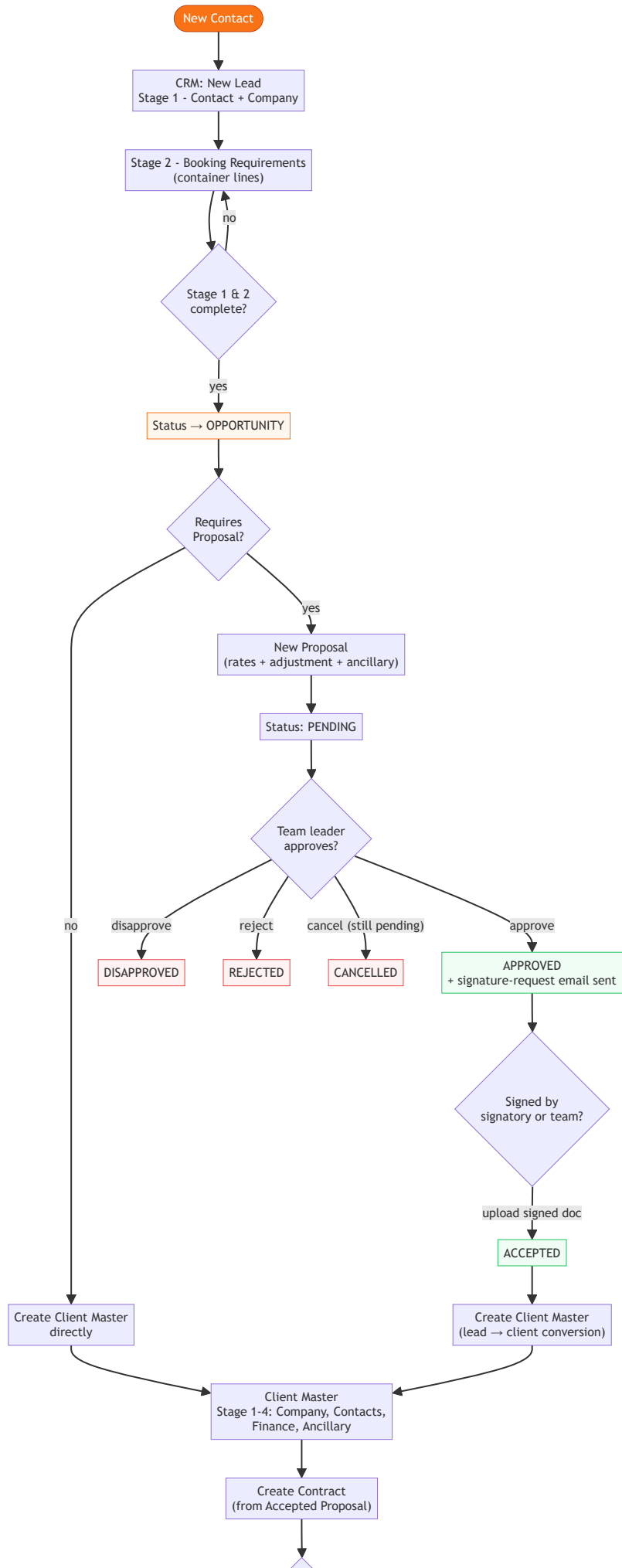
Every live module, grouped by domain. The gate chip shows how its mutating actions are protected — `permission: middleware`, `nav.access: middleware`, ad hoc team/role checks, or nothing beyond login.

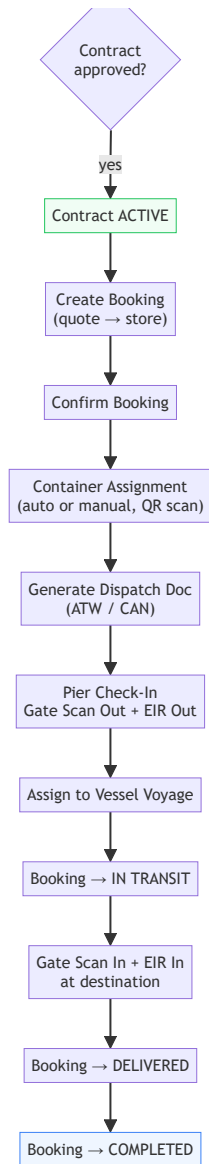


■ Sales & CRM ■ Operations ■ Fleet ■ Administration

2 • Business Process Flow

The core lifecycle a deal follows end to end — from a sales rep capturing a lead to cargo being delivered and the booking closed out.

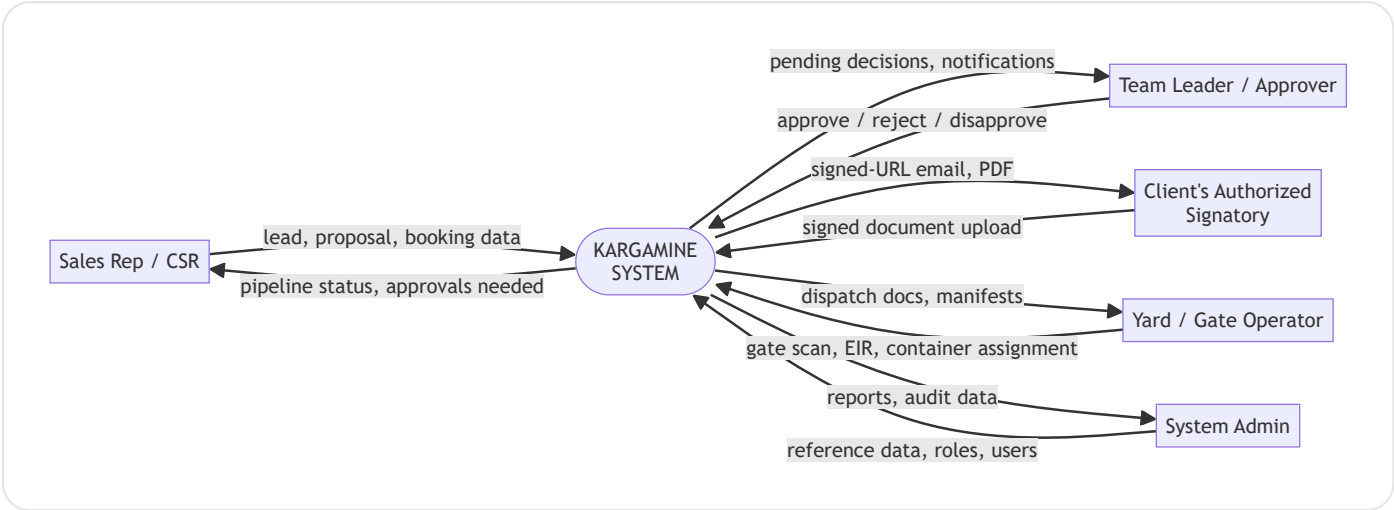




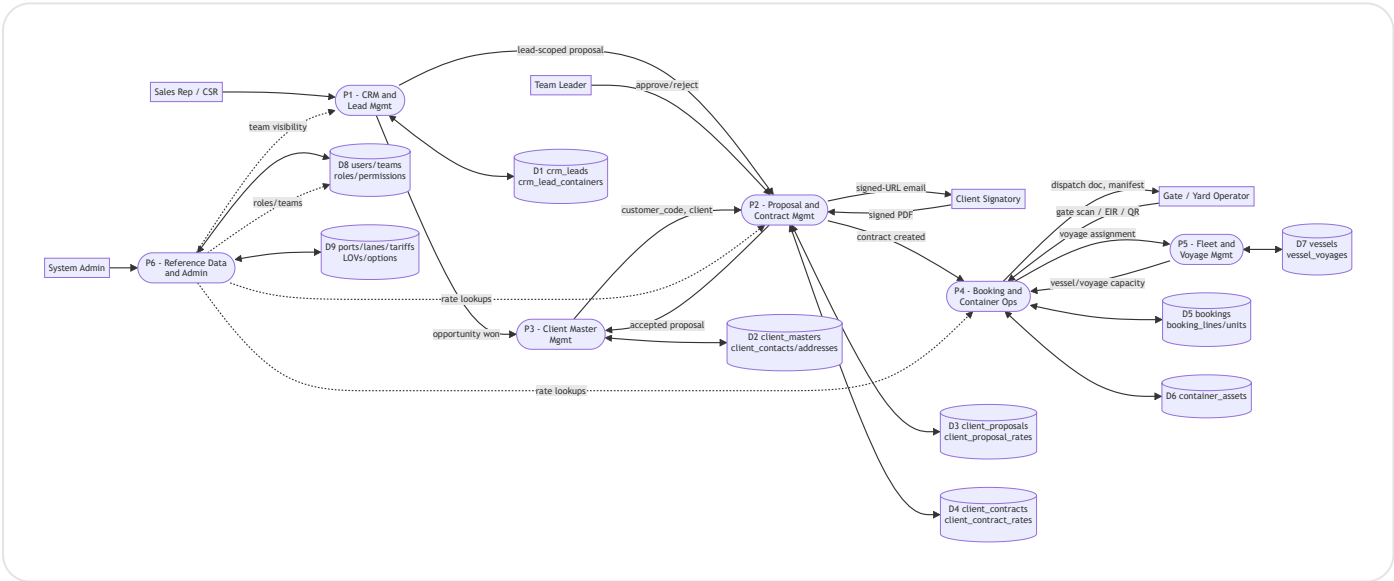
3 • Data Flow Diagram

Level 0 context, then a Level 1 breakdown into the six processes that make up the system. Cylinders are data stores (tables/table groups), rounded boxes are processes, plain rectangles are external entities.

LEVEL 0 — CONTEXT



LEVEL 1 — PROCESS BREAKDOWN



4 • Function Inventory

Every controller behind each module, with its public methods as actually declared in code.

1 • Dashboard

[/page_dashboard](#)

DashboardController

summary

2 • CRM (Leads)

[/page_crm](#) • [/page_crmLeadForm](#)

CrmLeadController

index assignableUsers saveStage1 saveStage2 storeContainer uploadDgDocument store show
getOrGenerateCustomerCode update destroy

CrmStatusController

index store show update destroy

CrmActivityController

index store show update destroy

CrmNoteController

store

3 • Proposals (Client Proposals)

[/page_proposals](#)

ClientProposalController

index indexAll indexByLead show rateLookup store storeForLead addRates destroyRate
leadContainerDefaults approve disapprove reject cancel attachSigned downloadPdf

ProposalSigningController

— public, signed-URL only, no auth

show downloadPdf upload

ProposalController

— legacy/parallel, pre-ClientProposal

index store createPdf getByCode processApprovals

4 • Clients (Client Masters)

[/page_clientMasters](#) • [/page_clientMasterForm](#)

ClientMasterController

index show saveStage1 saveStage2 saveStage3 saveStage4 destroy

5 · Contracts (Client Contracts)

[/page_contracts](#)

ClientContractController

[index](#) [indexAll](#) [show](#) [downloadPdf](#) [store](#) [createFromProposal](#) [approve](#) [terminate](#)

ContractController

— second/parallel controller; verify which the live UI calls before gating

[index](#) [show](#) [store](#) [ratesFromProposal](#) [update](#) [destroy](#)

6 · Booking / Bookings

[/page_booking](#) · [/page_bookingForm](#)

BookingController

[index](#) [show](#) [quote](#) [store](#) [update](#) [updateLineDetails](#) [uploadHazmatDocument](#) [confirm](#) [downloadBol](#) [cancel](#)
[markInTransit](#) [markDelivered](#) [markCompleted](#)

BookingDispatchController

[generate](#) [updateCvAssignment](#)

BookingEirController

[uploadFile](#) [issueOut](#) [issueIn](#)

BookingGateScanController

[pending](#) [scan](#)

BookingVoyageController

[assign](#) [shutOut](#)

7 · Cargo Build-Up

[/page_cargo_build_up](#)

CargoBuildUpController

[index](#) [bookings](#) [booking](#)

8 · Container Assignment

[/page_container_assignment](#)

ContainerAssignmentController

[index](#) [lookup](#) [assign](#) [unassign](#) [autoAssignBooking](#)

DeviceContainerAssignmentController

— dormant, Sanctum-token only, no browser session

[assign](#)

9 · Pier Check-In (Gate Scan)

[/page_pier_checkin](#)

BookingGateScanController

— shared with Booking module

[pending](#) [scan](#)

10 · Container Inventory (Container Assets)

/page_container_inventory

ContainerAssetController

- index
- show
- store
- update
- markUnderRepair
- markAvailable
- markOutOfService
- markDamaged
- relocate
- available
- reserve
- release

11 · Vessel Management

/page_vessel_management

VesselController

- index
- store
- show
- update
- changeStatus
- storeMaintenanceRecord
- destroyMaintenanceRecord

12 · Voyage Schedule

/page_voyage_schedule

VesselVoyageController

- index
- store
- show
- update
- destroy
- loadlist
- containersForVessel
- manifest

13 · Users

/page_users

UserController

- index
- create
- show
- store
- save_info
- deactivate
- reactivate
- getUserSettings
- byRole
- counts

RolesController — /page_roles_permissions, nav-less

- index
- store
- update
- destroy
- permissions

14 · Team Management

/page_team_management

TeamController

- index
- store
- update
- destroy
- members
- availableUsers
- addMember
- updateMember
- removeMember

Fourteen independent CRUD controllers over reference/rate data, each with the same index/store/show/update/destroy shape unless noted:

- PortController
- LocationController
- ChargeTypeController
- DeliveryTypeController
- ServiceableAreaController
- LaneController
- SpecialChargeController
- CargoYardController
- LaneTariffRateController
- PortChargeController
- GeneralChargeController
- HandlingFeeController
- TruckingTariffController
- VatRateController

ContainerController — catalog/variants, distinct from Container Inventory's physical assets

- index
- show
- store
- update
- destroy
- variants

ListOfValueController / OptionController — generic LOV admin, backs the LovController named lookups

- index
- store
- show
- update
- destroy
- byOption
- storeByOption

LovController — read-only named lookups consumed by dropdowns app-wide

- route
- service
- vantype
- vansize
- vanclass
- typeOfBusiness
- addressType
- leadSource
- cargoType
- industry
- industrySubcategory
- organizationType
- clientCategory
- clientClassification
- contactDepartment
- unit

15b · Developer Option

MailerController

- index
- save
- send
- test

MenusController

- index
- menulist
- store
- update
- destroy
- swapMenuOrder

AppThemeController

- show
- update

AppInformationController

- show
- update
- removeLogo
- removeIcon

NotificationController

- index
- unreadCount
- markRead
- testSend

Cross-cutting

ProfileController

- show
- update
- updateLayout
- updatePassword
- uploadPhoto
- deletePhoto

PageController — one method per Blade page shell; returns the fragment loadPage() swaps in

- page_dashboard
- page_crm
- page_proposals
- page_clientMasters
- page_contracts
- page_Booking
- page_CargoBuildUp
- page_ContainerAssignment
- page_PierCheckin
- page_ContainerInventory
- page_VesselManagement
- page_VoyageSchedule
- page_Users
- page_TeamManagement
- page_maintenance
- ... +19 more (see below)

5 • Detailed Step-by-Step Process

The same lifecycle as the flowchart above, written out as an operational checklist with the exact route/function each step calls.

1 Lead Capture (CRM)

1. Rep opens + **New Lead** → `crmLeadForm.blade.php`, fills Contact + Company (Stage 1) → `CrmLeadController::saveStage1`.
 2. Rep adds one or more Booking Requirement container lines (Stage 2) → `CrmLeadController::saveStage2` / `storeContainer` for single-line appends.
 3. `CrmLead::recomputeCompletion()` checks both stages; once both are complete the lead's status auto-promotes from `LEAD` / `QUALIFIED` to `OPPORTUNITY`.
 4. Optional: DG (dangerous goods) documentation uploaded per container line → `uploadDgDocument`.
-

2 Proposal Creation & Approval

1. If the lead's **Require Proposal** flag is checked, rep opens the New Proposal side-modal from the Lead Info view and adds container lines with Route, Container/Class/Size, Base Rate (auto-looked-up via `rateLookup`), Adjustment (discount/increase), Pickup/Drop-off Area, and optional Ancillary Services → `ClientProposalController::storeForLead` (lead-scoped) or `store` (client-scoped, once a Client Master already exists).
 2. Proposal saves as `PENDING`; the lead's direct team leader(s) are notified.
 3. The team leader over the lead's assigned rep — walking up through `TeamService::accessibleTeamIds` — reviews and either **Approves**, **Disapproves**, or **Rejects** it (reject is also open to the client's own Sales Rep). While still `PENDING`, the creator/approving leader can also **Cancel** it.
 4. On **Approve**, the system emails the client's Authorized Signatory (pulled from the lead's Company Info) a signed, unauthenticated link → `ProposalSigningController::show`, good for 30 days.
 5. The signatory downloads the approved PDF, signs it digitally or physically, and uploads the signed copy back through that same link → `ProposalSigningController::upload`, which flips status straight to `ACCEPTED`. (A team member can alternatively upload it from inside the app via `attachSigned`.)
-

3 Client Conversion

1. Once the lead has an accepted proposal (or **Require Proposal** was unchecked), the **Create Client Master** action becomes available → reserves/reuses the lead's `customer_code`, then walks the 4-stage Client Master wizard: Company Info & Addresses, Contacts, Finance, Ancillary Services → `ClientMasterController::saveStage1-4`.
 2. Any proposal still attached to the originating lead (`client_id` null) gets backfilled with the new `client_id` so it stops being lead-scoped-only.
 3. The lead's status flips to `WIN` as soon as a Client Master row exists at all, even before all 4 stages are filled in.
-

4 Contract

1. From an `ACCEPTED` proposal tied to a real Client Master, rep clicks **Create Contract** → `ClientContractController::createFromProposal`, which copies the proposal's rate lines into contract rate lines the rep can still adjust.
 2. Contract goes through its own **Approve** step → `approve`; once `ACTIVE` it's what bookings are actually priced against.
 3. A contract can later be **Terminated** → `terminate` (permission-gated: `contract.terminate`).
-

5 Booking

1. Ops creates a booking against the active contract → quote first (`quote`), then `store` (permission: `booking.create`).
 2. Booking line details can be edited before confirmation → `updateLineDetails` ; hazmat docs attach per line if needed.
 3. **Confirm** the booking → `confirm` (permission: `booking.confirm`) — locks it in as real cargo to move, not just a draft.
 4. A booking can be **Cancelled** at this stage → `cancel` (permission: `booking.cancel`).
-

6 Container Assignment & Dispatch

1. Physical containers are matched to booking lines — auto-assign remaining slots (`autoAssignBooking`) or manually via QR scan / typed ID (`assign`), enforcing Draft-only + release-before-reassign + container-type-match rules in `ContainerReservationService::assignToUnit()` .
 2. Generate the dispatch document (ATW/CAN) → `BookingDispatchController::generate` ; update ConVan/Proforma BL/Waybill/Seal assignment as needed → `updateCvAssignment` (permission: `booking.generate-dispatch-document` / `booking.assign-cv`).
-

7 Gate & Vessel Operations

1. Container passes Pier Check-In: gate scan out → `BookingGateScanController::scan` (permission: `booking.gate-scan`), then EIR Out issued → `BookingEirController::issueOut` (permission: `booking.issue-eir`).
 2. Assign the booking to a vessel voyage → `BookingVoyageController::assign` (permission: `booking.assign-voyage`); booking status advances to `IN TRANSIT` → `BookingController::markInTransit` .
 3. Loadlist/manifest generated for the voyage → `VesselVoyageController::loadlist` / `manifest` (permission: `booking.generate-loadlist`).
 4. At destination: gate scan in + EIR In issued → `scan` / `issueIn` .
-

8 Delivery & Close-Out

1. Booking marked **Delivered** → `markDelivered` , then **Completed** → `markCompleted` once all paperwork/billing is settled.
 2. Container asset released back to available inventory → `ContainerAssetController::release` , ready for the next assignment.
-

6 · Known Gaps & Orphaned Code

Straight from `MODULES.md` — real, documented architectural facts worth knowing before extending anything below.

Four uncoordinated authorization mechanisms coexist:

- `permission`: middleware — the modern system; only covers Booking, Contract create/terminate, and `roles.manage`.
- `nav.access`: middleware — covers most Settings-area mutations.
- Ad hoc role/team checks — `TeamService` hierarchy for CRM + Proposal approval, `config/client_proposal_workflow.php` reject roles, `can:isSuperAdmin` Gate, direct `role_name` string comparisons.
- Sanctum token abilities — brand new, only guards the dormant device-integration endpoint.

ITEM	STATUS
<code>routes/api_booking.php</code>	Never required — dead code
<code>routes/api_contracts.php</code>	Never required — dead code
<code>routes/api_master.php</code>	Never required — dead code
<code>routes/pageApi.php</code>	Never required — dead code
<code>/page_usermanagement</code>	Superseded by <code>/page_users</code> , kept as internal reference only
<code>/page_reports</code> , <code>/page_help</code> , <code>/page_profile</code> , <code>/settings</code>	Stubs / unfinished
7 <code>PageController</code> methods	No route at all (<code>page_Forms</code> , <code>page_featuredHome</code> , <code>page_documents</code> , <code>page_approvals</code> , <code>page_reports_documents</code> , <code>page_reports_users</code> , <code>page_finance_tracker</code>)
<code>ListingController</code> , <code>ActivityController</code> , <code>Api/MenuController</code>	No route references found anywhere